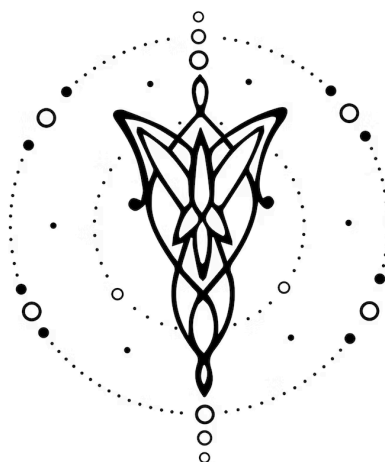




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Augusto	Cicciomessere	62267	acicciomessere@itba.edu.ar
Josefina	Miguez	63132	jmiguez@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en: [DocuJSON](#)

3. Dominio

Desarrollar un lenguaje que permita crear y personalizar documentación, con la idea de mostrar con claridad los métodos, las variables y los parámetros de los distintos elementos del código.

Para el desarrollo del lenguaje se va a codificar a partir de etiquetas en un formato similar al JSON, donde se aclaren los componentes del código y teniendo dentro de cada uno la posibilidad de darle una descripción y un estilo base a la documentación en sí.

En el caso que salga correctamente el código, este será un html con un apartado para el estilo, junto con la estructura correspondiente a lo especificado en la entrada del código, proporcionando finalmente una documentación correctamente estructurada.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Se podrán crear a partir de la etiqueta *methods* la documentación para cada función dentro del código, junto con sus parámetros, su salida y una breve descripción.
- (II). Se podrán crear variables con la etiqueta *variables* en la documentación las cuales serán necesarias para mayor comprensión del código ya que hacen referencia a variables globales dentro de las funciones.
- (III). Se podrá agregar un estilo al texto, ya sea el espaciado entre elementos (entre funciones por ejemplo), tipo de texto y tamaño.
- (IV). Las etiquetas son *methods*, *variables*, *style*, *params*, *description*, *related* y *type*.
- (V). Las etiquetas de *description* y *type* se utilizan dentro de *variables* y *methods*, *params* se utiliza dentro de *methods*.
- (VI). Para las etiquetas de *type* solo se soportan como tipo para variables o retorno de funciones string, boolean, int y char.
- (VII). No se proveerán operadores relacionales, ni aritméticos, y lógicos.
- (VIII). Se proveerán relaciones entre las distintas funciones con la etiqueta *related* (por ejemplo el caso que una utilice otra función en la documentación).

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que construya una documentación utilizando sólo funciones.
- (II). Un programa que construya una documentación utilizando sólo variables.
- (III). Un programa que construya una documentación con estilo y utilizando tanto variables como funciones.
- (IV). Un programa que construya una documentación con descripciones para las funciones.

- (V). Un programa que construya una documentación con funciones relacionadas entre sí.
- (VI). Un programa que implemente 2 o más funciones que se relacionan.
- (VII). Un programa que implemente variables y funciones con descripciones de cada elemento.
- (VIII). Un programa que construya documentación que tenga una función sin parámetros, variables y estilo.
- (IX). Un programa que construya documentación con una función recursiva.
- (X). Un programa que construya documentación con 2 funciones que se llamen entre sí (función 1 usa la función 2 y viceversa).

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa malformado.
- (II). Un programa que relacione una función que no existe a otra que si.
- (III). Un programa que no tenga ningún tipo de retorno soportado.
- (IV). Un programa que utilice estilo no soportado.
- (V). Un programa que tenga variables que tengan un tipo no soportado.

6. Ejemplos

-Creación de una documentación para manejar fechas:

```
{
  methods: {
    formatDate: {
      params: {
        date: {
          type: Date,
          description: "The date object to format."
        },
        format: {
          type: string,
          description: "The format string, e.g., 'YYYY-MM-DD' or 'DD/MM/YYYY'."
        }
      },
      description: "Formats a date object into a string according to the given pattern.",
      type: "string",
      variables: {
        year: {
          type: number,
          description: "Extracted year from the date object."
        },
        month: {
          type: number,
          description: "Extracted month from the date object, 0-indexed."
        },
        day: {
          type: number,
          description: "Extracted day of the month from the date object."
        }
      }
    }
  }
}
```

```

    },
    formatted: {
      type: string,
      description: "Final formatted string to return."
    }
  },
  parseDate: {
    params: {
      dateString: {
        type: string,
        description: "The date string to parse, e.g., '2025-04-04'."
      },
      format: {
        type: string,
        description: "The format the string is in, e.g., 'YYYY-MM-DD'."
      }
    },
    description: "Parses a date string into a Date object using the specified format.",
    type: "Date",
    variables: {
      formatParts: {
        type: array,
        description: "Array of format tokens (e.g., ['YYYY', 'MM', 'DD'])."
      },
      dateParts: {
        type: array,
        description: "Array of parts extracted from the date string (e.g., ['2025', '04', '04'])."
      },
      year: {
        type: number,
        description: "Parsed year as number."
      },
      month: {
        type: number,
        description: "Parsed month as number (0-indexed for Date object)."
      },
      day: {
        type: number,
        description: "Parsed day as number."
      }
    }
  },
  style: {
    methods: {
      title: "color: navy, font-size: 28px",
      description: "color: darkslategray, font-size: 16px"
    }
  }
}

```

Creación de documentación de una calculadora:

```
{
  methods: {
    add: {
      params: {
        a: {
          type: number,
          description: "The first number to be added."
        },
        b: {
          type: number,
          description: "The second number to be added."
        }
      },
      description: "Adds two numbers and returns the result.",
      type: "number"
    },
    subtract: {
      params: {
        a: {
          type: number,
          description: "The number to subtract from."
        },
        b: {
          type: number,
          description: "The number to subtract."
        }
      },
      description: "Subtracts the second number from the first.",
      type: "number"
    },
    multiply: {
      params: {
        a: {
          type: number,
          description: "The first number to multiply."
        },
        b: {
          type: number,
          description: "The second number to multiply."
        }
      },
      description: "Multiplies two numbers.",
      type: "number"
    },
    divide: {
      params: {
        a: {
          type: number,
          description: "The dividend (number to be divided)."
        },

```

```

    b: {
      type: number,
      description: "The divisor (number by which to divide). Must not be zero."
    }
  },
  description: "Divides the first number by the second. Internally uses 'multiply' and 'invert' to
calculate division.",
  type: "number",
  related: ["multiply", "invert"]
},
invert: {
  params: {
    x: {
      type: number,
      description: "The number to invert (1 / x).",
    }
  },
  description: "Returns the multiplicative inverse of a number.",
  type: "number"
},
clear: {
  params: {},
  description: "Clears the current result or input.",
  type: "void"
}
},
style: {
  methods: {
    title: "color: green, font-size: 30px",
    description: "color: gray, font-size: 18px"
  }
}
}

```